Correlation analysis with human data

2025-07-09

This is an R Markdown document.

For more details on using R Markdown see http://rmarkdown.rstudio.com.

Check for installed packages

```
requireNamespace("dplyr")
```

```
## Loading required namespace: dplyr
```

```
requireNamespace("ggplot2")
```

```
## Loading required namespace: ggplot2
```

```
requireNamespace("generics")
requireNamespace("purrr")
```

```
## Loading required namespace: purrr
```

```
requireNamespace("broom")
```

```
## Loading required namespace: broom
```

```
requireNamespace("ggrepel")
```

```
## Loading required namespace: ggrepel
```

Install any missing packages by uncommenting the corresponding line

```
#install.packages("dpylr")
#install.packages("ggplot2")
#install.packages("generics")
#install.packages("purrr")
#install.packages("broom")
#install.packages("ggrepel")
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(ggplot2)
library(purrr)
library(generics)
```

```
##
## Attaching package: 'generics'
```

```
## The following object is masked from 'package:dplyr':
##
##     explain
```

```
## The following objects are masked from 'package:base':
##
##     as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
##     setequal, union
```

```r
library(broom)
library(ggrepel)
```

```r
get_double_plot <- function(processed_base_data, processed_inst_data) {

  base_merged <- processed_base_data %>%
    left_join(human_judgment_by_group, by = c("title", "version")) %>%
    mutate(source = "Base")

  inst_merged <- processed_inst_data %>%
    left_join(human_judgment_by_group, by = c("title", "version")) %>%
    mutate(source = "Instruct")

  combined_data <- bind_rows(base_merged, inst_merged)

  regression_stats <- inst_merged %>%
    group_by(prompt_type) %>%
    summarise(
      model = list(lm(percent_yes ~ prob_diff)),
      .groups = "drop"
    ) %>%
    mutate(
      tidy_model = map(model, tidy),   # coefficient-level info
      glance_model = map(model, glance), # model-level info
      slope = map_dbl(model, ~ coef(.x)[["prob_diff"]]),
      r2 = map_dbl(glance_model, ~ .x$r.squared),
      slope_pval = map_dbl(tidy_model, ~ .x$p.value[.x$term == "prob_diff"]) # p-value for slope
    )

  regression_label1 <- regression_stats %>%
    transmute(
      prompt_type,
      label = sprintf("R² = %.3f", r2),
      x = -0.4,
      y = 0.5   # adjust depending on your y-scale
    )

  regression_label2 <- regression_stats %>%
```

```
    transmute(
      prompt_type,
      label = sprintf("p = %.3g", slope_pval),
      x = -0.42,
      y = 0.35   # adjust depending on y-scale
    )

  plot <- ggplot(combined_data, aes(x = prob_diff, y = percent_yes, color = source, shape = source)) +
    geom_point(alpha = 0.6, size = 2.5) +
    # Regression line for inst_data only
    geom_smooth(
      data = filter(combined_data, source == "Instruct"),
      method = "lm",
      se = FALSE,
      color = "black",
      linetype = "dashed"
    ) +
    geom_text(
      data = regression_label1,
      aes(x = x, y = y, label = label),
      inherit.aes = FALSE,
      hjust = 0,
      size = 3
    ) +
    geom_text(
      data = regression_label2,
      aes(x = x, y = y, label = label),
      inherit.aes = FALSE,
      hjust = 0,
      size = 3
    ) +
    facet_wrap(~ prompt_type, scales = "free") +
    labs(
      x = "p(covered) - p(uncovered)",
      y = "Human Judgement (%covered)",
      color = "Llama-70B",
      shape = "Llama-70B",
      title = "Human and LLM judgment by Question Variation"
    ) +
    theme_minimal()

  best_r2_row <- regression_stats %>% slice_max(r2, n = 1)

  return (list(
    plot=plot,
    best_prmopt_type=best_r2_row$prompt_type,
    inst_merged=inst_merged)
    )
}
```

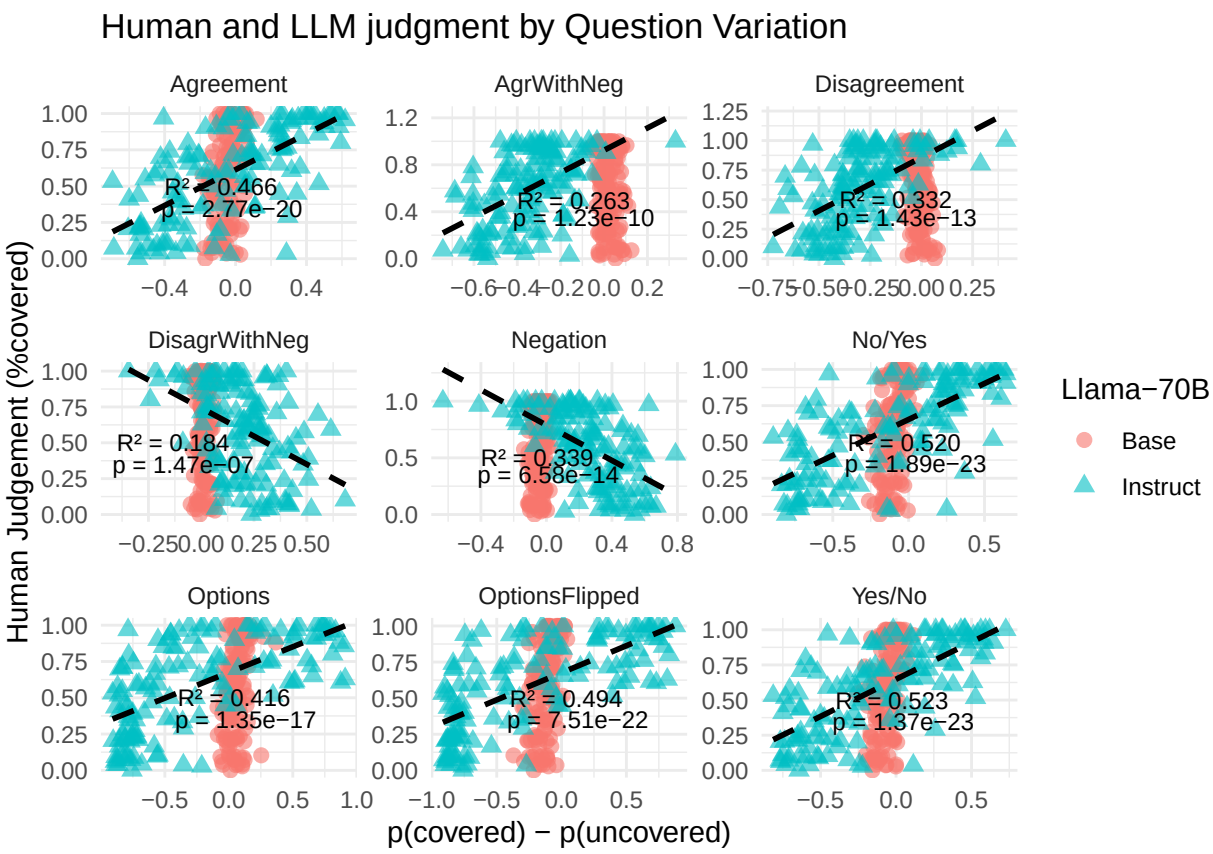
```
# Pipeline to generate Figure 3
setwd(".")
```

```r
# base_data_path <- ".\\runs\\runs-42_07_16\\meta-llama\\Llama-3.1-70B-results.csv"
base_data_path <- file.path("runs", "runs-42_07_16", "meta-llama", "Llama-3.1-70B-results.csv")
base_data <- read.csv(base_data_path)
#instruct_data_path <- ".\\runs\\runs-42_07_16\\meta-llama\\Llama-3.3-70B-Instruct-results.csv"
instruct_data_path <- file.path("runs", "runs-42_07_16", "meta-llama", "Llama-3.3-70B-Instruct-results.
instruct_data <- read.csv(instruct_data_path)
#human_judgement_path <- ".\\data\\human\\main-merged.csv"
human_judgement_path <- file.path("data", "human", "main-merged.csv")
human_data <- read.csv(human_judgement_path)

human_judgment_by_group <- get_human_judgment_by_group(human_data)
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```r
processed_base_data <- process_llm_data(base_data)
processed_inst_data <- process_llm_data(instruct_data)

result <- get_double_plot(processed_base_data, processed_inst_data)

plot <- result$plot
best_prompt_type <- result$best_prmopt_type
inst_merged <- result$inst_merged
plot
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



```r
ggsave("llama-human.pdf", plot, width = 6, height = 4, dpi = 300)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```r
get_single_plot <- function(processed_inst_data) {

  inst_merged <- processed_inst_data %>%
    left_join(human_judgment_by_group, by = c("title", "version")) %>%
    mutate(source = "Instruct")

  regression_stats <- inst_merged %>%
    group_by(prompt_type) %>%
    summarise(
      model = list(lm(percent_yes ~ prob_diff)),
      .groups = "drop"
    ) %>%
    mutate(
      tidy_model = map(model, tidy),    # coefficient-level info
      glance_model = map(model, glance), # model-level info
      slope = map_dbl(model, ~ coef(.x)[["prob_diff"]]),
      r2 = map_dbl(glance_model, ~ .x$r.squared),
      slope_pval = map_dbl(tidy_model, ~ .x$p.value[.x$term == "prob_diff"]) # p-value for slope
    )

  regression_label1 <- regression_stats %>%
    transmute(
      prompt_type,
      label = sprintf("R² = %.3f", r2),
      x = -0.4,
      y = 0.5   # adjust depending on your y-scale
    )

  regression_label2 <- regression_stats %>%
    transmute(
      prompt_type,
      label = sprintf("p = %.3g", slope_pval),
      x = -0.42,
      y = 0.35   # adjust depending on y-scale
    )

  plot <- ggplot(inst_merged, aes(x = prob_diff, y = percent_yes, color = source, shape = source)) +
    geom_point(alpha = 0.6, size = 2.5) +
    # Regression line for inst_data only
    geom_smooth(
      data = filter(inst_merged, source == "Instruct"),
      method = "lm",
      se = FALSE,
      color = "black",
      linetype = "dashed"
    ) +
    geom_text(
      data = regression_label1,
      aes(x = x, y = y, label = label),
      inherit.aes = FALSE,
```

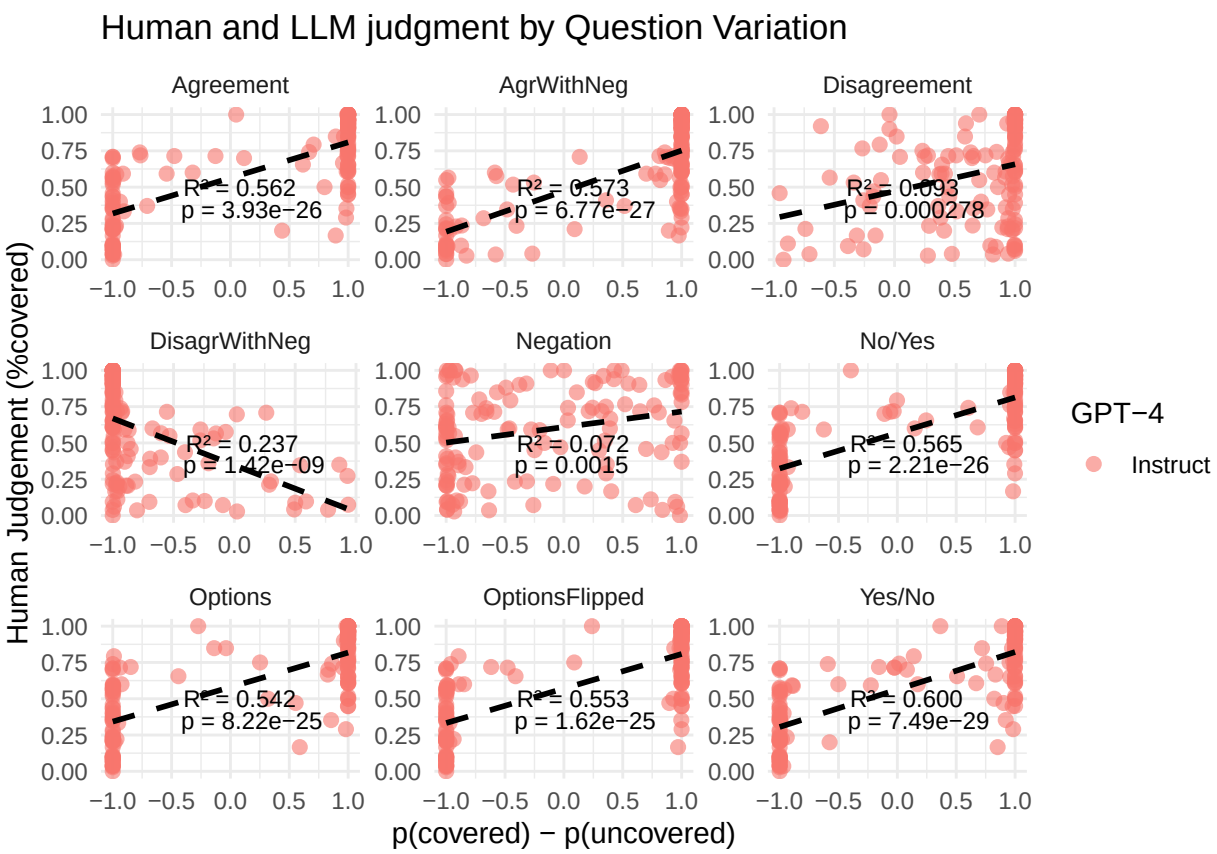
```
      hjust = 0,
      size = 3
    ) +
    geom_text(
      data = regression_label2,
      aes(x = x, y = y, label = label),
      inherit.aes = FALSE,
      hjust = 0,
      size = 3
    ) +
    facet_wrap(~ prompt_type, scales = "free") +
    labs(
      x = "p(covered) - p(uncovered)",
      y = "Human Judgement (%covered)",
      color = "GPT-4",
      shape = "GPT-4",
      title = "Human and LLM judgment by Question Variation"
    ) +
    theme_minimal()

  best_r2_row <- regression_stats %>% slice_max(r2, n = 1)

  return (list(
    plot=plot,
    best_prmopt_type=best_r2_row$prompt_type,
    inst_merged=inst_merged)
    )
}
```

```
# Pipeline to generate Figure 3 modified to generate Figure 5

# base_data_path <- "H:\\Documents\\Github\\llms-legal-interp\\runs\\runs-42_07_16\\meta-llama\\Llama-3
# base_data <- read.csv(base_data_path)
instruct_data_path <- file.path("runs", "runs-42_07_16", "openai-gpt-4-results.csv")
instruct_data <- read.csv(instruct_data_path)
#human_judgement_path <- ".\\data\\human\\main-merged.csv"
human_judgement_path <- file.path("data", "human", "main-merged.csv")
human_data <- read.csv(human_judgement_path)

human_judgment_by_group <- get_human_judgment_by_group(human_data)
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```
# processed_base_data <- process_llm_data(base_data)
processed_inst_data <- process_llm_data(instruct_data)

result <- get_single_plot(processed_inst_data)

plot <- result$plot
best_prompt_type <- result$best_prmopt_type
inst_merged <- result$inst_merged
plot
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

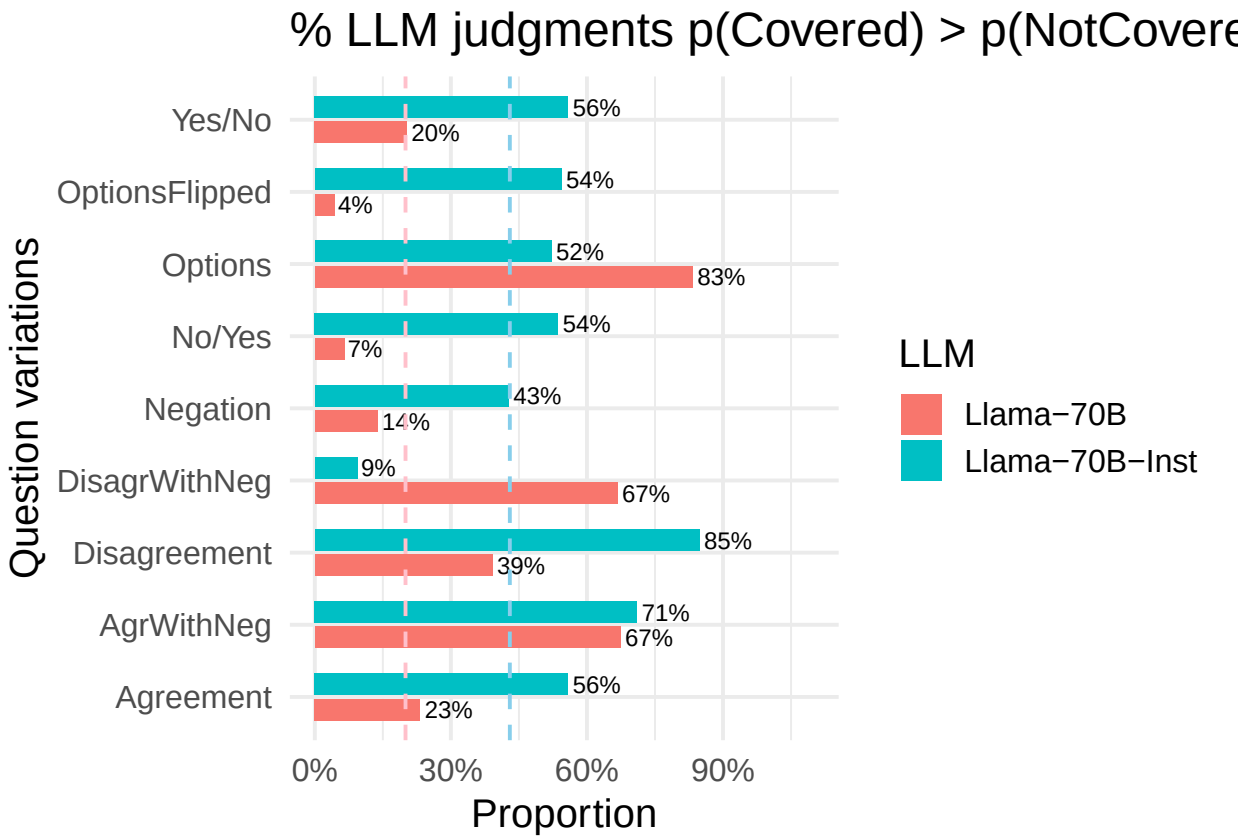



```
ggsave("gpt-human.pdf", plot, width = 6, height = 4, dpi = 300)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
# Pipeline to generate Figure 2, needs to run Figure 3 pipline first
get_response_direction <- function(df, source_name) {
  df %>%
    mutate(
      source = source_name,
      prob_positive = prob_diff > 0
    )
}

base_labeled <- get_response_direction(processed_base_data, "Llama-70B")
inst_labeled <- get_response_direction(processed_inst_data, "Llama-70B-Inst")

combined_labeled <- bind_rows(base_labeled, inst_labeled)

# Summarize proportion of positive responses
response_summary <- combined_labeled %>%
  group_by(prompt_type, source) %>%
  summarise(
    proportion_positive = mean(prob_positive, na.rm = TRUE),
    count = n(),
    .groups = "drop"
  )

# Bar plot
```

```
llama_pos <- ggplot(response_summary, aes(y = prompt_type, x = proportion_positive, fill = source)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_text(
    aes(label = scales::percent(proportion_positive, accuracy = 1)),
    position = position_dodge(width = 0.7),
    hjust = -0.1,
    size = 3
  ) +
  geom_vline(xintercept=0.20, linetype="dashed", color="pink") +
  geom_vline(xintercept=0.43, linetype="dashed", color="skyblue") +
  scale_x_continuous(labels = scales::percent_format(), limits = c(0, 1.1)) +
  labs(
    title = "% LLM judgments p(Covered) > p(NotCovered)",
    x = "Proportion",
    y = "Question variations",
    fill = "LLM"
  ) +
  theme_minimal(base_size=15)

llama_pos
```




```
# ggsave("llama-positive.pdf", plot = llama_pos, width = 7, height = 4, units = "in")
```

```
get_single_plot <- function(processed_data, human_judgment_by_group) {

  merged <- processed_data %>%
    left_join(human_judgment_by_group, by = c("title", "version")) %>%
```

```
    mutate(source = "Base")

  # Step 3: Prepare annotation data for plotting
  regression_stats <- merged %>%
    group_by(prompt_type) %>%
    summarise(
      model = list(lm(percent_yes ~ prob_diff)),
      .groups = "drop"
    ) %>%
    mutate(
      tidy_model = map(model, tidy),   # coefficient-level info
      glance_model = map(model, glance), # model-level info
      slope = map_dbl(model, ~ coef(.x)[["prob_diff"]]),
      r2 = map_dbl(glance_model, ~ .x$r.squared),
      slope_pval = map_dbl(tidy_model, ~ .x$p.value[.x$term == "prob_diff"]) # p-value for slope
    )

  # Step 4: Prepare annotation data for plotting

  regression_label1 <- regression_stats %>%
    transmute(
      prompt_type,
      label = sprintf("R² = %.3f", r2),
      x = -0.4,
      y = 0.5   # adjust depending on your y-scale
    )

  regression_label2 <- regression_stats %>%
    transmute(
      prompt_type,
      label = sprintf("p = %.3g", slope_pval),
      x = -0.42,
      y = 0.35   # adjust depending on your y-scale
    )

  best_r2_row <- regression_stats %>% slice_max(r2, n = 1)
  best_r2_data <- filter(merged, prompt_type==best_r2_row$prompt_type)

  return (list(
    regression_result=best_r2_row,
    model_data=best_r2_data)
    )
}
```

Threshold Analysis

```
threshold_analysis <- function(model_data, regression_result) {

  best_prompt <- regression_result$prompt_type
  highest_r2_data <- filter(model_data, prompt_type==best_prompt)
  model <- filter(regression_result, prompt_type==best_prompt)["model"][[1]][[1]]
  print(model)
  # threshold <- predict(model, data.frame(percent_yes=c(0.5)))
```

```
  threshold <- solve(model$coefficients[2], 0.5-model$coefficients[1])
  check <- predict(model, data.frame(prob_diff=c(threshold)))
  print(c(threshold, check))
  cat("Threshold: ", threshold)
  highest_r2_data$hum_judg <- with(highest_r2_data, ifelse(percent_yes > 0.5, TRUE, FALSE))
  highest_r2_data$hum_judg_strict <- with(highest_r2_data, ifelse(percent_yes > 0.7 | percent_yes < 0.3
  highest_r2_data$lin_thr_judg <- with(highest_r2_data, ifelse(prob_diff > threshold, TRUE, FALSE))
  highest_r2_data$lin_thr_acc <- with(highest_r2_data, ifelse(lin_thr_judg == hum_judg, TRUE, FALSE))
  lin_the_acc = sum(highest_r2_data$lin_thr_acc) / nrow(highest_r2_data)
  cat("; Linear threshold accuracy:", lin_the_acc)

  highest_r2_data_strict <- filter(highest_r2_data, version!="controversial" & hum_judg_strict==TRUE)

  lin_the_acc_str = sum(highest_r2_data_strict$lin_thr_acc) / nrow(highest_r2_data_strict)
  cat("; Strict threshold accuracy:", lin_the_acc_str)

  return(list(
    threshold=threshold,
    accuracy=lin_the_acc
  ))
}
```

```
inner_loop <- function(file_name){
  data <- read.csv(file_name)
#  human_judgement_path <- "H:\\Documents\\Github\\vague_contracts\\data\\1_falseconsensus\\main-merged
  human_judgement_path <- file.path("data", "human", "main-merged.csv")
  human_data <- read.csv(human_judgement_path)


  human_judgment_by_group <- get_human_judgment_by_group(human_data)
  processed_base_data <- process_llm_data(data)


  result <- get_single_plot(processed_base_data, human_judgment_by_group)


  regression_result <- result$regression_result
  model_data <- result$model_data

  last_part <- tail(strsplit(file_name, "\\\\")[[1]], 1)
  file_name_no_ext <- substr(last_part, 1, nchar(last_part) - 12)
  # model_prompt_pair <- cat(file_name_no_ext, "_", regression_result$prompt_type)

  model_data$model_name <- file_name_no_ext
  regression_result$model_name <- file_name_no_ext

  linear_pred_results <- threshold_analysis(model_data, regression_result)
  regression_result$threshold <- linear_pred_results$threshold
  regression_result$accuracy <- linear_pred_results$accuracy

  return(list(
    model_data=model_data,
    regression_result=regression_result
  ))
```

```
}
```

```
# Pipeline to generate and Best Prompt for Each Model Figure and Table 6
model_files <- c(
  "gpt2-medium-results.csv",
  "openai-gpt-4-results.csv",
  file.path("mistralai", "Ministral-8B-Instruct-2410-results.csv"),
  file.path("meta-llama", "Llama-3.1-70B-results.csv"),
  file.path("meta-llama",  "Llama-3.2-1B-results.csv"),
  file.path("meta-llama",  "Llama-3.2-1B-results.csv"),
  file.path("meta-llama",  "Llama-3.1-8B-Instruct-results.csv"),
  file.path("meta-llama",  "Llama-3.2-3B-Instruct-results.csv"),
  file.path("meta-llama",  "Llama-3.1-8B-results.csv"),
  file.path("meta-llama",  "Llama-3.2-3B-results.csv"),
  file.path("meta-llama",  "Llama-3.2-1B-Instruct-results.csv"),
  file.path("meta-llama",  "Llama-3.3-70B-Instruct-results.csv"),
  file.path("allenai",  "OLMo-2-1124-7B-Instruct-results.csv"),
  file.path("allenai",  "OLMo-2-1124-7B-results.csv"),
  file.path("google",  "gemma-7b-it-results.csv"),
  file.path("google",  "gemma-7b-results.csv")
)

model_datas <- list()
regression_results <- list()

for (i in seq_along(model_files)) {
  #prefix <- "H:\\Documents\\Github\\llms-legal-interp\\runs\\runs-42_07_16\\"
  full_path <- file.path("runs", "runs-42_07_16", model_files[i])

  loop_results <- inner_loop(full_path)

  model_data <- loop_results$model_data
  regression_result <- loop_results$regression_result

  model_datas[[i]] <- model_data
  regression_results[[i]] <- regression_result


}
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.5803       0.4294
##
##                     1
## -0.1871375  0.5000000
## Threshold:  -0.1871375; Linear threshold accuracy: 0.6231884; Strict threshold accuracy: 0.5921053
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.5637       0.2582
##
##                     1
## -0.2465003  0.5000000
## Threshold:  -0.2465003; Linear threshold accuracy: 0.8333333; Strict threshold accuracy: 0.9736842
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.6023       0.9745
##
##                     1
## -0.1050232  0.5000000
## Threshold:  -0.1050232; Linear threshold accuracy: 0.6884058; Strict threshold accuracy: 0.8157895
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.8344       1.9834
##
##                     1
## -0.1686137  0.5000000
## Threshold:  -0.1686137; Linear threshold accuracy: 0.6884058; Strict threshold accuracy: 0.7763158
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.6189      -0.9103
##
##                   1
```

```
## 0.1306549 0.5000000
## Threshold:  0.1306549; Linear threshold accuracy: 0.3768116; Strict threshold accuracy: 0.4078947
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.6189      -0.9103
##
##                    1
## 0.1306549 0.5000000
## Threshold:  0.1306549; Linear threshold accuracy: 0.3768116; Strict threshold accuracy: 0.4078947
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.9087       1.1355
##
##                      1
## -0.3599176  0.5000000
## Threshold:  -0.3599176; Linear threshold accuracy: 0.6666667; Strict threshold accuracy: 0.75
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.6133       0.4601
##
##                      1
## -0.2463381  0.5000000
## Threshold:  -0.2463381; Linear threshold accuracy: 0.5869565; Strict threshold accuracy: 0.6052632
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.5272       1.0775
```

```
##
##                       1
## -0.02528448  0.50000000
## Threshold:  -0.02528448; Linear threshold accuracy: 0.5797101; Strict threshold accuracy: 0.5789474
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.5735      -0.6745
##
##                    1
## 0.1089313 0.5000000
## Threshold:  0.1089313; Linear threshold accuracy: 0.4202899; Strict threshold accuracy: 0.4078947
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.6259      -0.3716
##
##                    1
## 0.3386964 0.5000000
## Threshold:  0.3386964; Linear threshold accuracy: 0.3768116; Strict threshold accuracy: 0.4078947
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.6482       0.5289
##
##                    1
## -0.280177  0.500000
## Threshold:  -0.280177; Linear threshold accuracy: 0.7898551; Strict threshold accuracy: 0.9210526
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
```

```
## (Intercept)    prob_diff
##      0.6265       0.2006
##
##                     1
## -0.6304192  0.5000000
## Threshold:  -0.6304192; Linear threshold accuracy: 0.6449275; Strict threshold accuracy: 0.7236842
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.6279       1.2998
##
##                       1
## -0.09839304  0.50000000
## Threshold:  -0.09839304; Linear threshold accuracy: 0.6449275; Strict threshold accuracy: 0.6447368
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.5338       0.2127
##
##                     1
## -0.1589507  0.5000000
## Threshold:  -0.1589507; Linear threshold accuracy: 0.6014493; Strict threshold accuracy: 0.5921053
```

```
## `summarise()` has grouped output by 'title'. You can override using the
## `.groups` argument.
```

```
##
## Call:
## lm(formula = percent_yes ~ prob_diff)
##
## Coefficients:
## (Intercept)    prob_diff
##      0.6396       1.3831
##
##                     1
## -0.1009563  0.5000000
## Threshold:  -0.1009563; Linear threshold accuracy: 0.5942029; Strict threshold accuracy: 0.6447368
```

```
final_model_data <- dplyr::bind_rows(model_datas)
final_reg_results <- dplyr::bind_rows(regression_results)

regression_label1 <- final_reg_results %>%
  transmute(
```

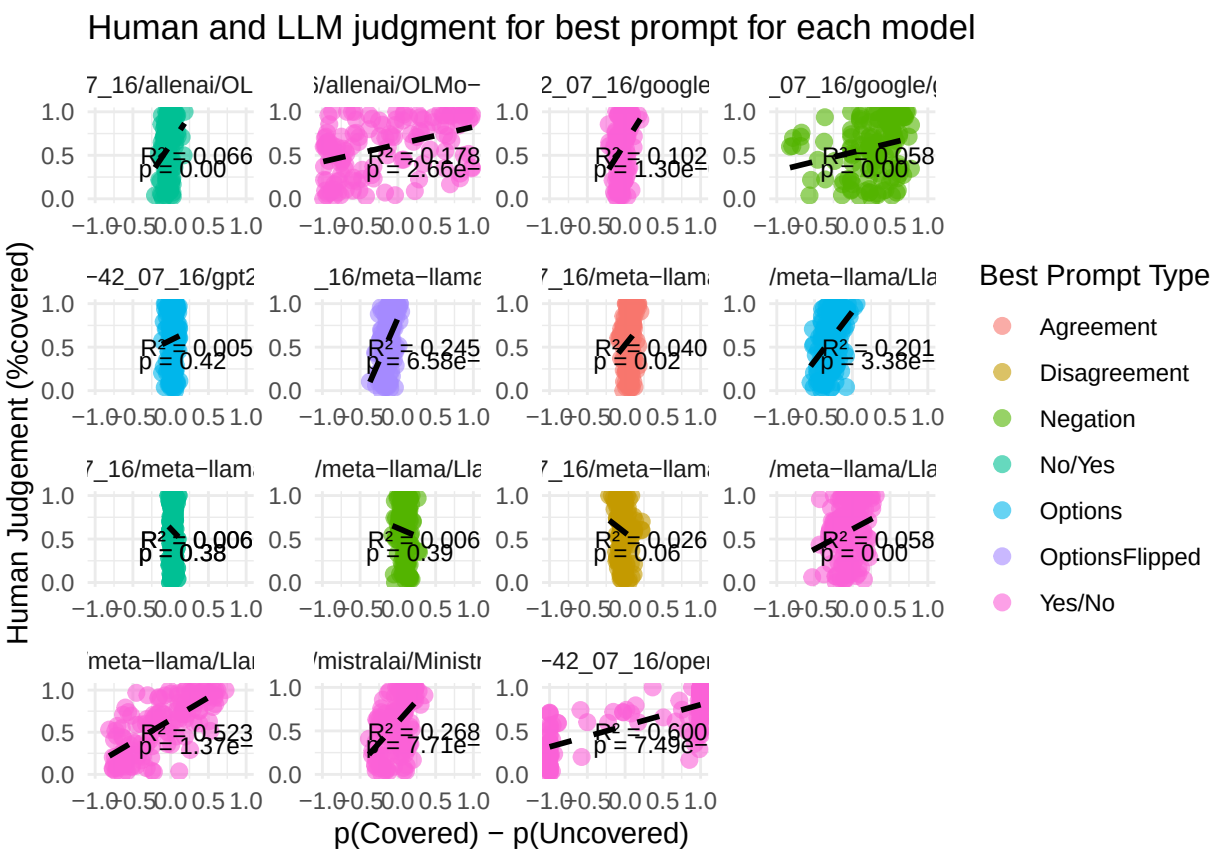
```
    model_name,
    label = sprintf("R² = %.3f", r2),
    x = -0.4,
    y = 0.5   # adjust depending on your y-scale
  )

format_p <- function(p) {
  ifelse(
    p >= 0.001,
    formatC(p, format = "f", digits = 2),
    formatC(p, format = "e", digits = 2)
  )
}

regression_label2 <- final_reg_results %>%
  transmute(
    model_name,
    label = sprintf("p = %s", format_p(slope_pval)),
    x = -0.42,
    y = 0.35   # adjust depending on your y-scale
  )


plot <- ggplot(final_model_data, aes(x = prob_diff, y = percent_yes, color = prompt_type)) +
  geom_point(alpha = 0.6, size = 2.5) +
  geom_smooth(
    data = final_model_data,
    method = "lm",
    se = FALSE,
    color = "black",
    linetype = "dashed"
  ) +
  geom_text(
    data = regression_label1,
    aes(x = x, y = y, label = label),
    inherit.aes = FALSE,
    hjust = 0,
    size = 3
  ) +
  geom_text(
    data = regression_label2,
    aes(x = x, y = y, label = label),
    inherit.aes = FALSE,
    hjust = 0,
    size = 3
  ) +
  facet_wrap(~ model_name, scales = "free") +
  labs(
    x = "p(Covered) - p(Uncovered)",
    y = "Human Judgement (%covered)",
    title = "Human and LLM judgment for best prompt for each model",
    color = "Best Prompt Type"
  ) +
```

```
  scale_x_continuous(limits = c(-1, 1)) +
  scale_y_continuous(breaks = c(0, 0.5, 1)) +
  theme_minimal()

plot
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 24 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 24 rows containing missing values or values outside the scale range
## (`geom_point()`).
```




```
final_reg_results # Table 6
```

```
## # A tibble: 16 x 10
##    prompt_type    model  tidy_model       glance_model  slope      r2 slope_pval
##    <chr>          <list> <list>           <list>        <dbl>   <dbl>      <dbl>
##  1 Options        <lm>   <tibble [2 x 5]> <tibble>      0.429 0.00477   4.21e- 1
##  2 Yes/No         <lm>   <tibble [2 x 5]> <tibble>      0.258 0.600     7.49e-29
##  3 Yes/No         <lm>   <tibble [2 x 5]> <tibble>      0.974 0.268     7.71e-11
##  4 OptionsFlipped <lm>   <tibble [2 x 5]> <tibble>      1.98  0.245     6.58e-10
##  5 No/Yes         <lm>   <tibble [2 x 5]> <tibble>     -0.910 0.00559   3.84e- 1
##  6 No/Yes         <lm>   <tibble [2 x 5]> <tibble>     -0.910 0.00559   3.84e- 1
##  7 Options        <lm>   <tibble [2 x 5]> <tibble>      1.14  0.201     3.38e- 8
##  8 Yes/No         <lm>   <tibble [2 x 5]> <tibble>      0.460 0.0577    4.56e- 3
##  9 Agreement      <lm>   <tibble [2 x 5]> <tibble>      1.08  0.0403    1.83e- 2
```

```
## 10 Disagreement   <lm>   <tibble [2 x 5]> <tibble>     -0.675 0.0257    6.05e- 2
## 11 Negation       <lm>   <tibble [2 x 5]> <tibble>     -0.372 0.00554   3.85e- 1
## 12 Yes/No         <lm>   <tibble [2 x 5]> <tibble>      0.529 0.523     1.37e-23
## 13 Yes/No         <lm>   <tibble [2 x 5]> <tibble>      0.201 0.178     2.66e- 7
## 14 No/Yes         <lm>   <tibble [2 x 5]> <tibble>      1.30  0.0659    2.38e- 3
## 15 Negation       <lm>   <tibble [2 x 5]> <tibble>      0.213 0.0585    4.27e- 3
## 16 Yes/No         <lm>   <tibble [2 x 5]> <tibble>      1.38  0.102     1.30e- 4
## # i 3 more variables: model_name <chr>, threshold <dbl>, accuracy <dbl>
```

```
ggsave("best_prompt_for_each_model.pdf", plot, width = 10, height = 7.5, dpi = 300)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 24 rows containing non-finite outside the scale range
## (`stat_smooth()`).
## Removed 24 rows containing missing values or values outside the scale range
## (`geom_point()`).
```